Postgres Job Queue

A persistent, at-least-once job queue built on Spring Boot and PostgreSQL. No broker, no Redis, no queue daemon. The `jobs` table *is* the queue.

This implements the semantics BullMQ provides, on Postgres, from scratch: atomic claiming under concurrent workers, lease-based failure detection, heartbeat renewal, bounded retry with exponential backoff, and dead-lettering.

The contract

Everything below is a limitation. A caller must know all three.

At-least-once delivery. A job may execute more than once. If a worker becomes unresponsive after claiming a job, that job is reclaimed and re-run. There is no way to prevent this: a worker that has gone silent may be dead or may be slow, and the two are indistinguishable from outside. **Job handlers must be idempotent.**

Claim-ordered, completion-unordered. Jobs are claimed in submission order (`created_at` ascending). Completion order is arbitrary, because N workers run in parallel. If job B must not run before job A completes, that is a caller-side dependency, not something this queue guarantees. Submit them as one job, or submit B only after observing A succeed.

Durability is Postgres's. A job that receives a `201` from `POST /jobs` is committed to disk. It survives the crash of every worker, the API, and the machine. Nothing is held in memory.

Architecture

Three actors. One table. Nothing pushes; workers poll.

```
  Client
    | POST /jobs {type, payload}
    v
  Spring API  ---- INSERT ---->  jobs (Postgres)  <---- claim / update
----  Worker 1..N
    |  201 {id}                        ^
(thread pool)
    v                                  |
  Client                               +---- scan / reset ----  Reaper
(@Scheduled)
```

The API inserts one row as `pending` and returns immediately. It never waits for a worker, and it has no idea whether one exists. `GET /jobs/{id}` reads the row's current status.

A worker is a `Runnable`, one per thread, on a fixed `ExecutorService`. Its loop: claim a row atomically, look up the handler registered for the job's `type`, call it with the payload, record the outcome. Repeat.

The reaper is a `@Scheduled` bean running a single `UPDATE` every few seconds. It finds rows whose lease has expired and returns them to `pending`.

Workers do not talk to each other, to the API, or to the reaper. All coordination is the database.

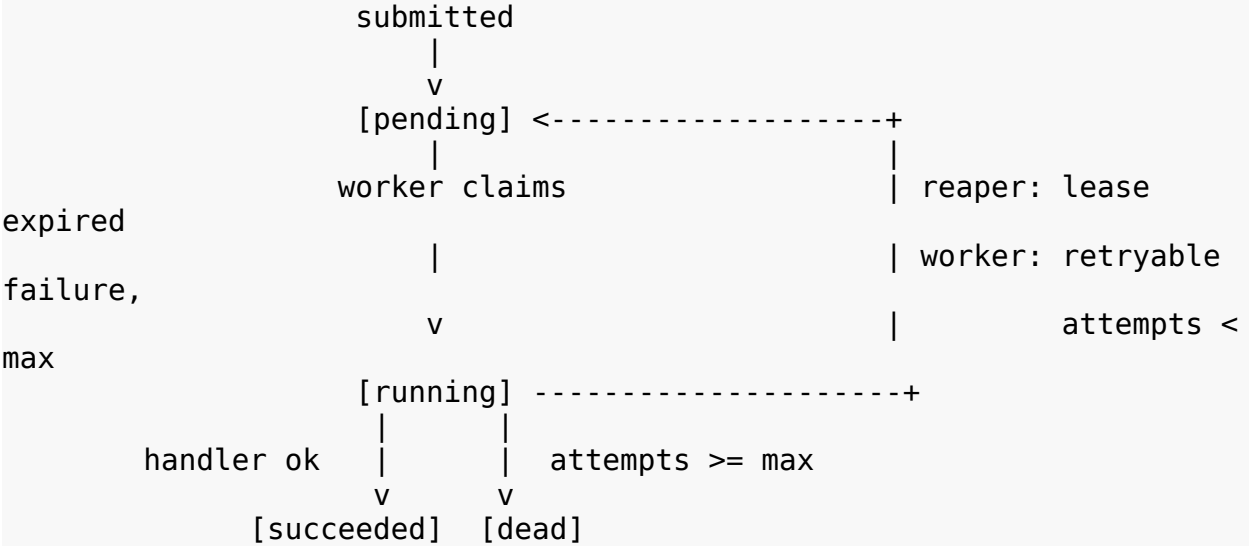
Data access

Schema lives in a Flyway migration. Queries are hand-written SQL executed through Spring's `JdbcTemplate`. There is no ORM: every query in this system depends on its exact `WHERE` clause and on the row count it returns, and an ORM writes the first and swallows the second. See DESIGN.md.

Endpoints

Method	Path	Body	Returns
`POST`	`/jobs`	`{"type": "...", "payload": {...}}`	`201` + `{"id": "<uuid>"}`
`GET`	`/jobs/{id}`	—	`200` + job status, or `404`

State machine



Writers, and what each is permitted to do:

Transition	Written by	Guarded by
`-> pending`	API (insert)	—
`pending -> running`	worker	`FOR UPDATE SKIP LOCKED`

Transition	Written by	Guarded by
`running -> succeeded`	worker	guard clause
`running -> pending`	worker (retry) or reaper (expired lease)	guard clause / predicate
`running -> dead`	worker or reaper, when `attempts >= max`	guard clause / predicate

The guard clause on every terminal write is:

```
WHERE id = ? AND worker_id = ? AND status = 'running'
```

A worker may believe it still owns a job that the reaper has already reclaimed and handed to someone else. This clause makes that worker's write match zero rows. It checks its row count, learns it lost the lease, logs, and abandons.

The heartbeat is an **optimization**: it makes the reaper's wrong guesses rare. The guard clause is **correctness**: it makes wrong guesses harmless. These are not the same mechanism and neither substitutes for the other.

Failure model

Four of these are detect-and-respond. One is prevented outright. One is a non-event, and it is in the table on purpose — knowing when the system should do *nothing* is part of the design.

Failure	Mechanism	Detection	Response
Two workers claim the same row simultaneously	prevented	n/a	`SELECT ... FOR UPDATE SKIP LOCKED` makes it structurally impossible. The claim and the exclusion are one statement.
Worker process dies mid-job (SIGKILL, OOM, power loss)	detected	The reaper's `WHERE status = 'running' AND lease_expires_at < now()`. Nothing detects the process. The reaper has no concept of a worker; it sees only an expired lease.	Row returns to `pending`, `attempts` incremented. Incrementing here, and not only on a thrown exception, is what stops a job that reliably OOM-kills its worker from cycling through the entire fleet forever.
Poison job: handler throws every time	detected	The handler throws. The worker catches	`attempts` incremented,

Failure	Mechanism	Detection	Response
		it.	`run_after` set to `now() + backoff`, status back to `pending`. Once `attempts >= max`, status becomes `dead`. A poison job never succeeds; there is no other branch.
Wedged handler: process alive, handler will never return	detected	The worker's own absolute per-job timeout fires. The reaper cannot help here: it can edit rows, not kill a thread on another machine.	Worker abandons the handler, stops heartbeating, marks the job failed under its guard clause, `attempts` incremented.
Slow but healthy job runs longer than its lease	none	n/a	Nothing fires. The heartbeat keeps moving `lease_expires_at` forward, so the reaper's predicate never matches. Knowing when the reaper should stay quiet is part of the design.
Worker writes an outcome for a job it no longer owns	detected	The guard clause `WHERE id = ? AND worker_id = ? AND status = 'running'` matches zero rows. The check is part of the write.	Worker logs the lost lease and abandons its result. The row is untouched, still owned by whoever holds it now.

A GC pause can starve a live worker's heartbeat thread, and the reaper will reclaim a job that was never abandoned. The reaper's guess can be wrong. Rows 2 and 6 are a pair: the heartbeat makes wrong guesses rare, the guard clause makes them harmless.

Why Postgres and not Redis

Redis gives you a blocking pop (`BRPOP`) — a worker holds a connection open and receives a job the instant one arrives, with no polling. It also gives atomicity for free, because Redis is single-

threaded. Postgres gives you neither: workers must poll, so a job waits up to one poll interval before pickup, and atomic claiming requires `SKIP LOCKED`.

What Postgres buys back is the thing that matters:

One transaction spanning the job's real work and the mark of completion. If a handler writes to the same Postgres instance, both the business write and the `status = 'succeeded'` write commit or roll back together. With Redis holding job state and Postgres holding business data, no transaction spans both. That is a dual write, and it cannot be made atomic. You will complete the work, fail to ack, and run the job again.

It also removes an entire piece of infrastructure from the deployment.

Systems built this way in production: Oban (Elixir), Solid Queue (Rails), River (Go).

Handlers

A handler is one method:

```
public interface JobHandler {
    String type();
    void handle(JsonNode payload) throws Exception;
}
```

The handler receives **the payload and nothing else.** Not `attempts`, not `created_at`, not the lease. Those are the queue's bookkeeping. Hand a handler `attempts` and retry policy immediately starts living in two places.

`type` is a routing key. Spring wires every `JobHandler` implementation into a map keyed by `type()`. `payload` is an opaque `jsonb` blob; the queue never parses it. Adding a job type means adding a class. The queue does not change.

Transient vs permanent failure

Only transient failures should be retried. A downstream timeout might succeed on the next attempt. A malformed payload will not.

The worker cannot tell them apart from a bare `Exception`, so the handler communicates it:

```
throw new TransientJobException("payment gateway timed out");  // retry
throw new PermanentJobException("payload missing orderId");    // dead-letter now
```

The judgment lives inside the handler, where the knowledge is.

One case the queue classifies on its own: a job whose `type` has no registered handler. That check happens before any handler runs, and it is never transient — every worker holds the same map. Dead-letter it on attempt 1.

Concurrency, in one sentence

Three separate mechanisms in this system share one shape:

- `FOR UPDATE SKIP LOCKED` — the claim
- `WHERE id = ? AND worker_id = ?` — the guard clause
- `WHERE status = 'running' AND lease_expires_at < now()` — the reaper

Make the check part of the write, not a step before it. A `SELECT` followed by an `UPDATE` is a race. A single `UPDATE` whose `WHERE` clause encodes the precondition is not, because Postgres serializes writers to a row and re-evaluates the predicate against the committed value.

Consequence: **the reaper is stateless and idempotent.** Two reapers hitting the same expired row do not corrupt it — the second one's predicate is false against the value the first one committed, and it updates zero rows. So N app instances can each run a reaper with no leader election, no lock table, no coordination.

You get this for free by writing the query correctly, not by adding machinery.

What this deliberately does not do

Ordering between jobs. Adding a `depends_on` column and gating claims on it is the first step toward a workflow engine — Airflow, Temporal, Step Functions. Failure cascades, diamond dependencies, and cycles all follow. That is a different product.

The right next primitive is a `partition_key`: a worker may hold at most one job per key at a time, so jobs sharing a key serialize while different keys stay parallel. This is what Kafka partitions do. It is roughly a hundred lines and it is not built here.

Low-latency pickup. Workers poll. A job submitted just after a poll waits for the next one. `LISTEN/NOTIFY` closes most of that gap and is the obvious v2.

Killing a wedged thread. `Thread.stop()` is unsafe and `interrupt()` is only a request — a thread blocked in a socket read with no timeout ignores it. The worker abandons the *job* and frees its slot; the thread may stay leaked. The real fixes are pushing timeouts down to every I/O call, or running handlers in a separate process where SIGKILL is available. This is why serious runtimes isolate handlers by process.

Exactly-once execution. Impossible. See the contract.

Build order

Each step ends in a test that fails before the step is written. Concurrency bugs do not announce themselves by running the app and looking at it.

1. Table, `POST /jobs`, `GET /jobs/{id}`. No workers. Verify rows land.
2. One worker, one thread, a handler that sleeps. No lease, no retry.
3. **Ten workers.** Submit 1000 jobs; assert each ran exactly once. This is where `SKIP LOCKED` earns its keep, and where a wrong claim query fails loudly.
4. **Lease and reaper.** `kill -9` a worker mid-job; assert the job completes on another.
5. **Heartbeat.** A job longer than its lease survives.
6. **Retry, backoff, dead-letter.** A handler that always throws lands in `dead` after N attempts.
7. **Handler timeout.** A handler that never returns is abandoned.

Build it as one application. Split the API and workers with Spring `@Profile` as the *last* step — the split is nearly free, because the two halves only ever communicated through the table. It also makes the demo good: `docker compose up --scale worker=5`, `docker kill` one, watch the reaper.

Running it

```
docker compose up

curl -X POST localhost:8080/jobs \
  -H 'Content-Type: application/json' \
  -d '{"type":"send-receipt","payload":{"userId":42,"orderId":918}}'

curl localhost:8080/jobs/<uuid>
```